CYBERSECURITY HACKATHON • TRACK 1: AGENTIC CYBER DEFENCE

Post-Quantum Cryptography (PQC) Secure-Code Reviewer Agent

An Autonomous, State-Machine-Driven Security Agent for Detecting Quantum-Vulnerable Cryptography, Generating FIPS 203/204 Patches, and Enforcing Zero-Trust Human Governance.

👤 **Role:** DevSecOps & AI Systems Architect

🎯 **Standards:** NIST FIPS 203 (ML-KEM) & FIPS 204 (ML-DSA)

🔒 **Safety:** LangGraph interrupt() HITL Gate

🚨 The Threat: "Harvest Now, Decrypt Later" (HNDL)

Why classical encryption is already failing against nation-state adversaries

THREAT MODELING

💥 Shor's Algorithm

Runs on Cryptographically Relevant Quantum Computers (CRQCs) to factor large integers and compute discrete logs in **polynomial time $\mathcal{O}(n^3)$**.

- **RSA (2048/4096)** ➔ Broken
- **ECDSA / ECDH** ➔ Broken
- **DSA / Diffie-Hellman** ➔ Broken

⚡ Grover's Algorithm

Provides a quadratic speedup $\mathcal{O}(\sqrt{N})$ for searching unstructured spaces, **halving effective symmetric key lengths**.

- **AES-128** drops to 64-bit security (Unsafe)
- **AES-256** retains 128-bit quantum security
- **MD5 / SHA-1** broken classically & quantum

🏢 Enterprise Reality

Organizations have millions of lines of legacy code with hardcoded cryptographic calls.

- Adversaries are **stockpiling encrypted data today** to decrypt retroactively.
- Manual code auditing takes months and lacks provable audit trails.
- NIST released FIPS 203/204 in August 2024—codebases must migrate now.

🗺️ Complete Repository Anatomy & File Guide

Modular structure: Presentation, Agent Core, Testbench, and DevSecOps

FILE STRUCTURE

🖥️ Presentation

- `app.py`: Streamlit dark dashboard (Port 8501) with side-by-side diffs.
- `.streamlit/config.toml`: Theme styling & server config.

🧠 Agent Engine

- `pqc_agent/state.py`: AgentState TypedDict & Pydantic models.
- `pqc_agent/tools.py`: AST scanner & FIPS patch generator.
- `pqc_agent/graph.py`: 5-node LangGraph state machine.
- `pqc_agent/audit.py`: SHA-256 recursive hash logger.

⚙️ DevSecOps & CI

- `pqc_agent/cli.py`: Headless CLI scanner for CI/CD.
- `pqc_config.yaml`: Policy rules & target algorithms.
- `.github/workflows/`: CI tests & Bandit SAST scans.
- `Dockerfile`: Hardened non-root container.

🧪 Testbench

- `sample_files/`: Vulnerable Python & Java crypto samples.
- `test_pqc_agent.py`: Unit tests for all scanner tools.
- `test_graph_e2e.py`: E2E HITL interrupt & resume test.

🧠 The LangGraph 5-Node State Machine & HITL Gate

Zero-Trust Safety: AI can propose code, but cannot touch production without human sign-off

AGENTIC AI ARCHITECTURE



🔍 Explainable AI & Uncertainty (XAI)

Rather than opaque black-box responses, the agent produces step-by-step Chain-of-Thought (CoT) reasoning. It explicitly isolates test fixtures from security-critical paths and states confidence metrics (0.95 for direct imports, 0.85 for call sites).

🛑 Non-Negotiable Human-in-the-Loop Barrier

Built with LangGraph's native `interrupt()` primitive. Execution freezes completely at Node 4 until a human reviewer reviews the side-by-side diff and clicks **Approve** or **Reject** via `Command(resume= … )`.

🔄 NIST Post-Quantum Cryptography Migration Matrix

CRYPTOGRAPHIC STANDARDS

Mapping legacy algorithms to the new NIST FIPS standards finalized in August 2024

Legacy Algorithm	Category	Vulnerability Reason	Target NIST Replacement	Standard Reference
RSA-2048 / 4096	Key Encapsulation / Signing	Shor's Algorithm (Integer Factorization)	**ML-KEM-768** / **ML-DSA-65**	FIPS 203 / FIPS 204
ECDH / DH	Key Exchange	Shor's Algorithm (Discrete Logarithm / ECDLP)	**ML-KEM-768** (Module Lattice KEM)	FIPS 203
ECDSA / DSA / EdDSA	Digital Signature	Shor's Algorithm (Elliptic Curve Discrete Log)	**ML-DSA-65** (Module Lattice Signature)	FIPS 204
DES / 3DES / Blowfish	Symmetric Cipher	Broken Classically & Grover's Speedup	**AES-256-GCM** (Authenticated Encryption)	NIST SP 800-38D
AES-128 (CBC/ECB)	Symmetric Cipher	Grover's reduces key strength to 64 bits	**AES-256-GCM** (Maintains 128-bit quantum security)	NIST SP 800-38D
MD5 / SHA-1	Hashing Function	Known Classical Collisions + Grover	**SHA-3-256 / SHA-512** (Keccak sponge)	FIPS 202

*Also supports hybrid classical/quantum pairings (e.g., `X25519_ML-KEM-768`) for phased backwards-compatible infrastructure transitions.

Dual Operation Modes: Web Dashboard & DevSecOps CLI

SYSTEM IMPLEMENTATION

Built for both human security architects and automated continuous integration pipelines

Mode A: Interactive Web UI (`app.py`)

Designed for cryptographers and security auditors exploring repositories:

- Live file upload or sample selector with line-numbered preview.
- Visual cards detailing line numbers, severity, and Shor/Grover risk.
- Side-by-side syntax diff viewer highlighting legacy code vs PQC patch.
- Approve / Reject decision buttons unblocking the state machine.
- One-click download of JSON & TXT audit trails with SHA-256 hashes.

Mode B: Headless CLI Engine (`pqc_agent/cli.py`)

Designed for automated CI/CD runners and pre-commit hooks:

- Reads policy thresholds from `pqc_config.yaml`.
- Scans repositories recursively without requiring UI or LLM tokens.
- Enforces build gates via standardized exit codes:
  - `Exit 0`: Clean code (Build passes).
  - `Exit 1`: Policy violation ≥ threshold (Build fails).
- Auto-generates FIPS 203/204 patches via `--generate-patches`.

🔒 Cryptographic Tamper-Evident Auditing (SHA-256)

GOVERNANCE & COMPLIANCE

Blockchain-style hash chaining guarantees non-repudiation for SOC 2 and NIST SP 800-53

⛓️ Recursive Hash Chaining Formula

Every audit entry cryptographically seals the previous state:

```
Hash_0 = SHA256("0" * 64) [Genesis]
Hash_i = SHA256(CanonicalJSON(Entry_i) + Hash_{i-1})
```

If an insider or adversary tampers with any past finding, severity rating, or approval timestamp, the computed terminal hash immediately breaks.

📜 Regulatory Compliance Mapping

- **NIST SP 800-53 Rev. 5 (SC-13):** Cryptographic protection against quantum attacks via migration to FIPS 203/204.
- **NIST SP 800-53 Rev. 5 (AU-10):** Non-repudiation enforced via the SHA-256 chain hash.
- **SOC 2 Type II (CC6.1 / CC6.6):** Strict Human-in-the-Loop authorization before code modification.
- **NSA CNSA 2.0:** Prepares enterprise software for national post-quantum mandates.

🚀 Live Demonstration Cheatsheet & Summary

Everything you need to run, test, and explain the project to friends and judges

DEMO GUIDE

Essential Commands

Web UI	`streamlit run app.py`
CLI Scan	`python -m pqc_agent.cli --target sample_files/`
Unit Tests	`python test_pqc_agent.py`
E2E HITL Test	`python test_graph_e2e.py`
Docker Compose	`docker compose up -d --build`

🏆 Key Pitch Takeaways for Friends

- **Not just a linter:** It reasons with an LLM, explains Shor vs Grover, and writes FIPS 203/204 code.
- **Zero-Trust safety:** The AI is mathematically blocked from auto-committing via LangGraph's `interrupt()`.
- **Full Production DevSecOps:** Shipped with Docker, GitHub Actions CI/CD, pre-commit hooks, and tamper-evident SHA-256 auditing.